**Exp-1:**

**Design Zero-shot, One-shot, and Few-shot prompts to generate a product description for a Smart Fitness Watch.**

**Aim:**

To implement and compare the effectiveness of Zero-shot, One-shot, and Few-shot promptingstrategies using a Large Language Model (LLM) API, and to analyze their impact on the accuracy and consistency of responses for a sentiment classification task.

**Algorithm :**

1. Start.
2. Import the required libraries (openai) and configure the API key.
3. Define the test input text on which sentiment classification is to be performed.
4. Construct a Zero-shot prompt containing only the task instruction, with no examples.
5. Construct a One-shot prompt containing the task instruction along with a single labeledexample.
6. Construct a Few-shot prompt containing the task instruction along with multiple (three or more) labeled examples.
7. Send each of the three prompts to the LLM API using the same test input and the same model parameters.
8. Capture and store the response generated for each prompting strategy.
9. Compare the three responses based on correctness, consistency, and format adherence.
10. Display the comparative results to the user.
11. Stop.

**Python Program:**

```
import openai

openai.api_key="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"

test_review = "The movie's pacing was slow but the acting was brilliant."

def call_llm(prompt):
        response = openai.ChatCompletion.create(
                        model="gpt-3.5-turbo",
                        messages=[{"role": "user", "content": prompt}],
                        temperature=0 )
return response["choices"][0]["message"]["content"].strip()
```

Zero-shot prompt

zero_shot_prompt = f"""Classify the sentiment of the following review as

Positive, Negative, or Neutral.

Review: {test_review}

Sentiment:"""

One-shot prompt one_shot_prompt = f"""Classify the sentiment of the review as Positive, Negative, or Neutral.

Review: "The food was cold and the service was terrible." Sentiment: Negative Review: {test_review}

Sentiment:"""

Few-shot prompt few_shot_prompt = f"""Classify the sentiment of the review as

Positive, Negative, or Neutral.

Review: "I loved every moment of this film." Sentiment: Positive

Review: "The food was cold and the service was terrible."

Sentiment: Negative Review: "The product works as described, nothing special."

Sentiment: Neutral Review: {test_review} Sentiment:"""

print("Zero-shot Result :", call_llm(zero_shot_prompt))

print("One-shot Result :", call_llm(one_shot_prompt))

print("Few-shot Result :", call_llm(few_shot_prompt))

**Input Test sentence:**

"The movie's pacing was slow but the acting was brilliant."

**Output Zero-shot Result:**

Neutral One-shot Result: Mixed / Neutral Few-shot Result: Neutral (leaning slightly positive)

**Result :**

The Zero-shot, One-shot, and Few-shot prompting strategies were successfully implementedandcompared. It was observed that the Zero-shot prompt executed fastest but gave a less consistent classification, the One-shot prompt improved output format consistency, and the Few-shot prompt produced the most accurate and stable classification because the model had more contextual examples to generalize from. Hence, Few-shot prompting was found to be the most effective strategy for this task.